Rather than using a single static reward probability distribution over *n* actions, like our original bandit problem, the contextual bandit simulator sets up a different reward distribution over the actions for each state. That is, we will have *n* different softmax reward distributions over actions for each of *n* states. Hence, we need to learn the relationship between the states and their respective reward distributions, and then learn which action has the highest probability for a given state.

As with all of our projects in this book, we'll be using PyTorch to build the neural network. In this case, we're going to build a two-layer feedforward neural network that uses rectified linear units (ReLU) as the activation function. The first layer accepts a 10-element one-hot (also known as 1-of-K, where all elements but one are 0) encoded vector of the state, and the final layer returns a 10-element vector representing the predicted reward for each action given the state.

Figure 2.6 shows the forward pass of the algorithm we've described. Unlike the lookup table approach, our neural network agent will learn to predict the rewards that each action will result in for a given state. Then we use the softmax function to give us a probability distribution over the actions and sample from this distribution to choose an arm (advertisement). Choosing an arm will give us a reward, which we will use to train our neural network.

Figure 2.6. A computational graph for a simple 10-armed contextual bandit. The `get_state()` function returns a state value, which is transformed into a one-hot vector that becomes the input data for a two-layer neural network. The output of the neural network is the predicted reward for each possible action, which is a dense vector that is run through a softmax to sample an action from the resulting probability distribution over the actions. The chosen action will return a reward and updates the state of the environment. θ_1 and θ_2 represent the weight parameters for each layer. The $\mathbb{N}$, $\mathbb{R}$, and $\mathbb{P}$ symbols denote the natural numbers (0, 1, 2, 3, ...), the real numbers (a floating-point number, for our purposes), and a probability, respectively. The superscript indicates the length of the vector, so $\mathbb{P}^{10}$ represents a 10-element vector where each element is a probability (such that all the elements sum to 1).